

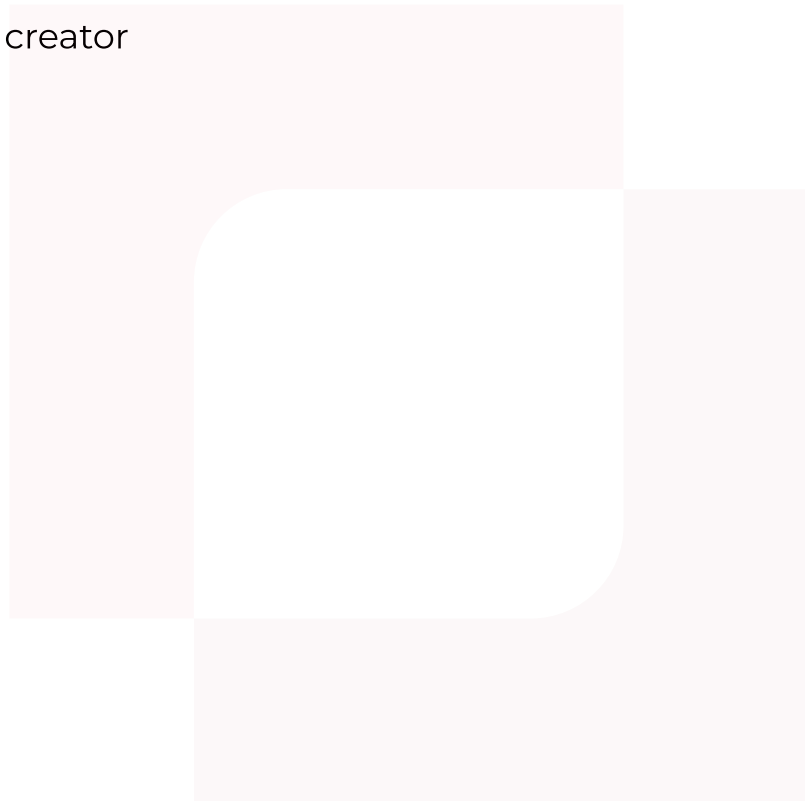
Using Type Erasure to Extend APIs You Don't Own

A Case Study From Audio Plugin Development

JAN WILCZEK

Who am I?

- Jan Wilczek [Yan Vil-check]
- Founder of TheWolfSound.com & online course creator
- C++ Speaker at think-cell





- Booking engine
- Self-service purchase, se

Age Group	Number of People
18-24	10
25-34	15
35-44	20

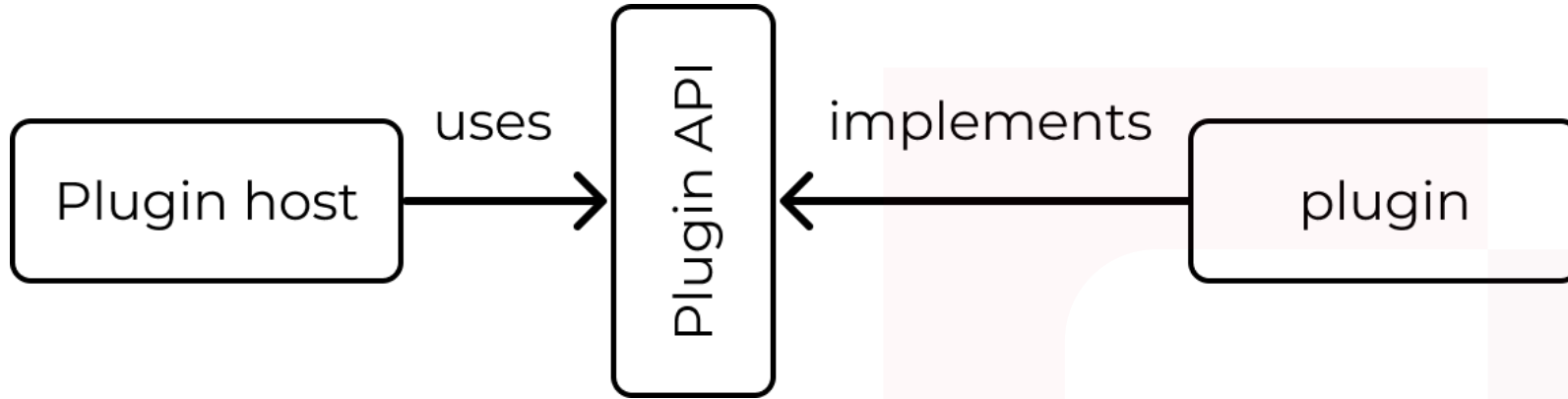
Create management-level charts and presentations

Who knows what audio plugins for digital audio workstations are?

Who has heard about the JUCE C++ framework?



Audio plugin formats



Audio plugin formats

reverb

VST3 plugin

reverb

AUv3 plugin

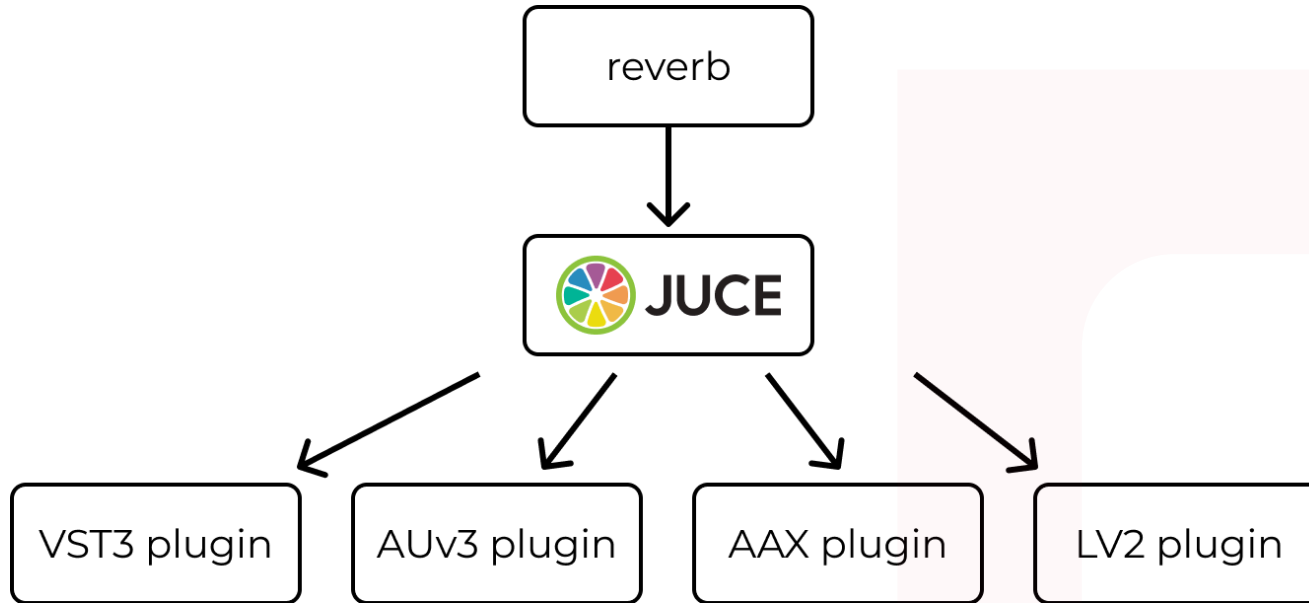
reverb

AAX plugin

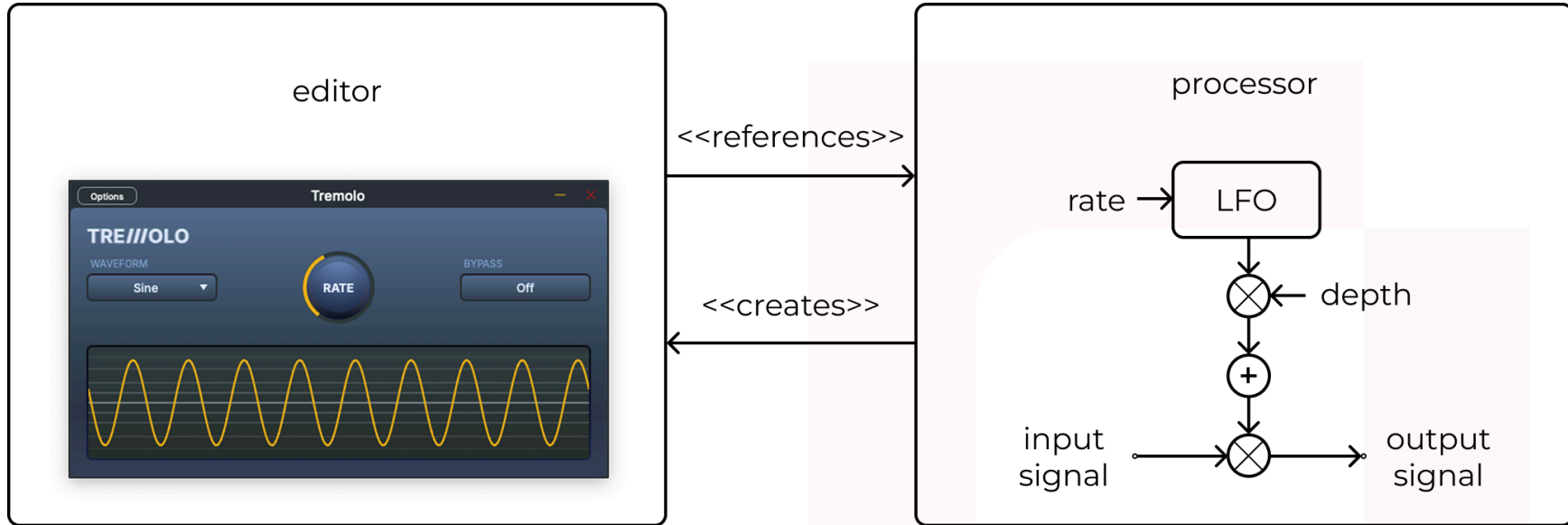
reverb

LV2 plugin

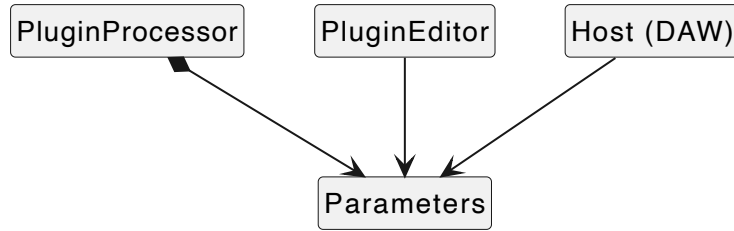
JUCE C++ framework



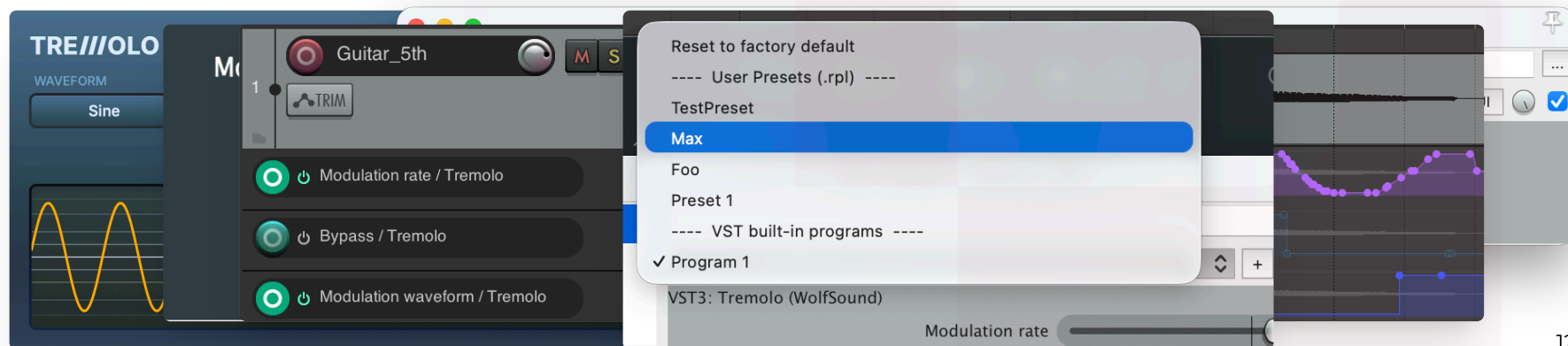
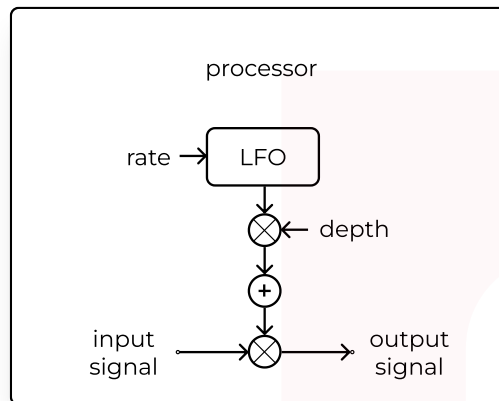
Plugin processor and editor



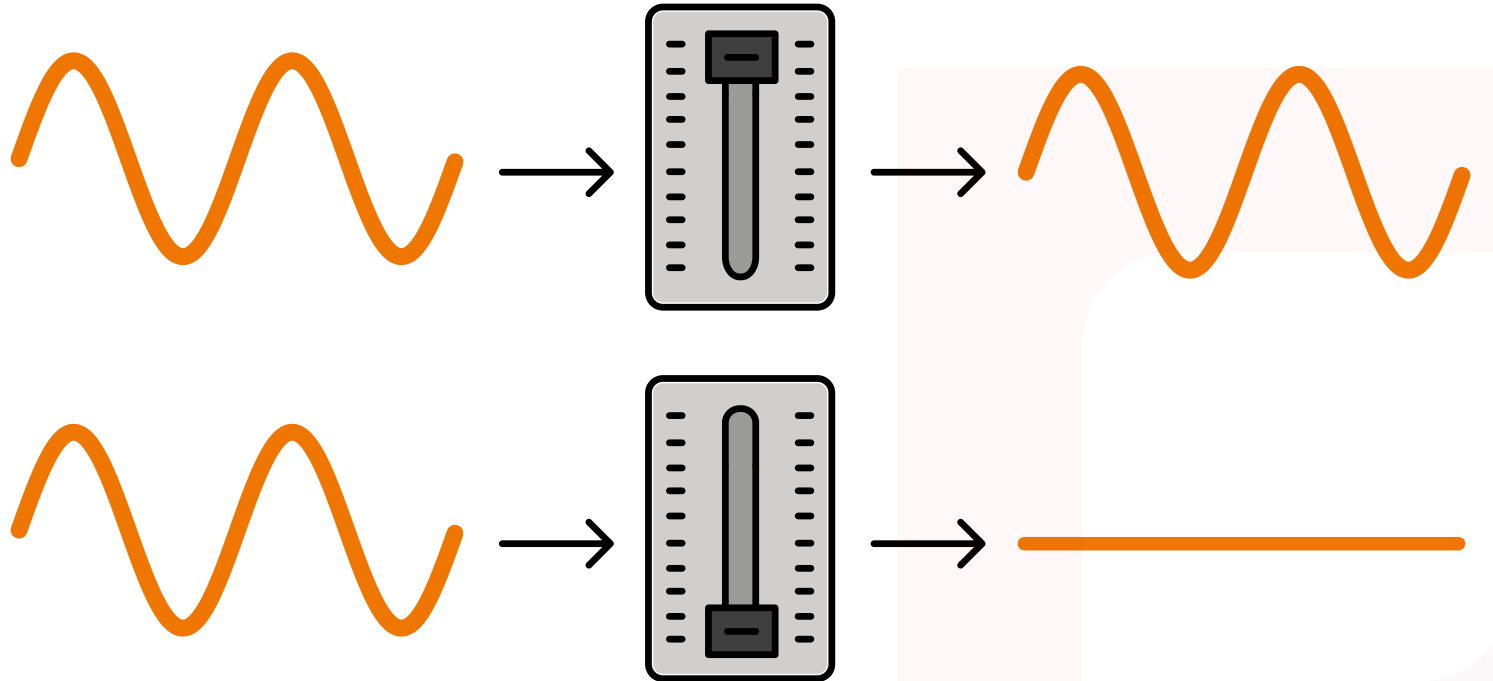
Parameters in audio plugins



Example of plugin parameters in action

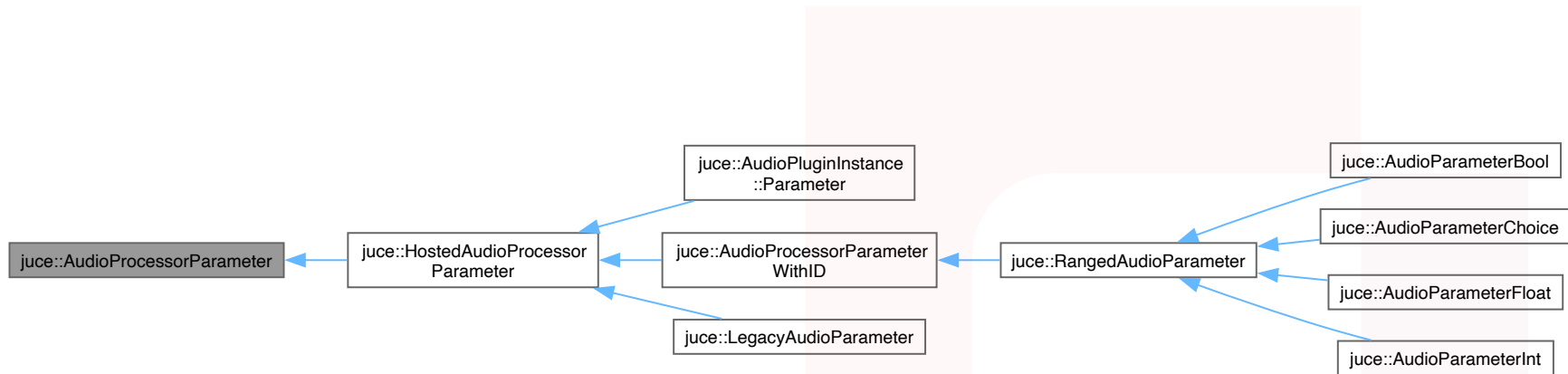


Gain parameter example



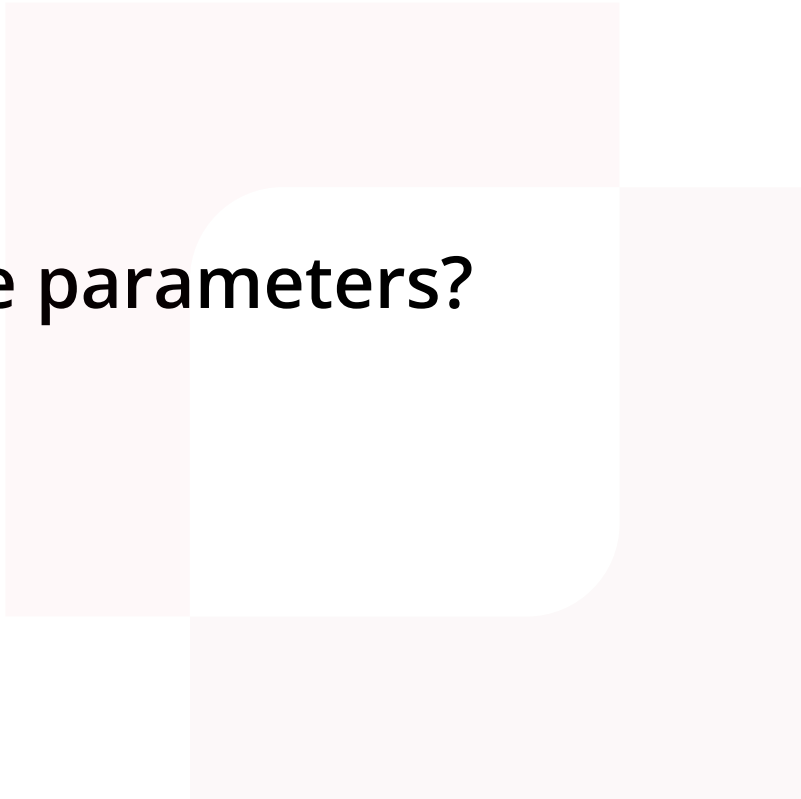
"level slider" by Sina Schulz from openmoji.org licensed under CC BY-SA 4.0, modified

Parameter class hierarchy in JUCE



JUCE parameter classes

- `AudioParameterBool`
 - `true / false`
- `AudioParameterInt`
 - integer in a closed range
 - e.g., "1" from {0, 1, 2}
- `AudioParameterFloat`
 - real value from a closed range
 - e.g., "0.25" from [0, 2]
- `AudioParameterChoice`
 - a value from a fixed set of named options
 - e.g., "lowpass" from {"lowpass", "highpass"}

A decorative graphic consisting of two overlapping squares. The top-left square is light red, and the bottom-right square is light pink. In the center, where they overlap, is a white rounded square.

How do we use parameters?

Plugin processor

```

1  class PluginProcessor : public AudioProcessor {
2  public:
3      PluginProcessor();
4      ~PluginProcessor();
5
6      void prepareToPlay(double sampleRate, int samplesPerBlock) override;
7      void releaseResources() override;
8
9      #ifndef JUCEPlugin_PreferredChannelConfigurations
10     bool isBusesLayoutSupported(const BusesLayout& layouts) const override;
11     #endif
12
13     void processBlock(AudioBuffer<float>&, MidiBuffer&) override;
14
15     AudioProcessorEditor* createEditor() override;
16     bool hasEditor() const override;
17
18     const String getName() const override;
19
20     bool acceptsMidi() const override;
21     bool producesMidi() const override;
22     bool isMidiEffect() const override;
23     double getTailLengthSeconds() const override;
24
25     int getNumPrograms() override;
26     int getCurrentProgram() override;
27     void setCurrentProgram(int index) override;
28     const String getProgramName(int index) override;
29     void changeProgramName(int index, const String& newName) override;
30
31     void getStateInformation(MemoryBlock& destData) override;
32     void setStateInformation(const void* data, int sizeInBytes) override;
33
34     juce::AudioParameterFloat& gain;
35
36 private:
37     JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR(PluginProcessor)
38 };

```

Parameters via references only

```
1  class PluginProcessor : public juce::AudioProcessor {  
2  public:  
3      PluginProcessor();  
4  
5      juce::AudioParameterFloat& gain;  
6  
7      // ...  
8  };
```

Parameters via references only

```
1 namespace {
2     juce::AudioParameterFloat& createGainParameter(juce::AudioProcessor& processor) {
3         auto parameter = std::make_unique<juce::AudioParameterFloat>("gain", "Gain", juce::NormalisableRange<float>{0.f, 1.f, 0.01f}, 1.f);
4         auto& result = *parameter;
5         processor.addParameter(parameter.release());
6         return result;
7     }
8 } // namespace
9
10
11 PluginProcessor::PluginProcessor() : // ...
12     gain{createGainParameter(*this)} {}
```

Parameters via references only

Usage in audio processing

```
1 void PluginProcessor::processBlock(juce::AudioBuffer<float>& buffer,  
2                                   juce::MidiBuffer&) {  
3     // ...  
4     buffer.applyGain(gain.get());  
5 }
```

Parameters via references only

Usage in UI

```
1  class PluginEditor : public juce::AudioProcessorEditor {
2  public:
3      explicit PluginEditor(PluginProcessor&);
4      // ...
5  private:
6      juce::Slider gainSlider;
7      juce::SliderParameterAttachment gainAttachment;
8  };
```

Parameters via references only

Usage in UI

```
1 PluginEditor::PluginEditor(PluginProcessor& p)
2     : AudioProcessorEditor(&p),
3       gainAttachment{p.gain, gainButton} {}
```

Parameters via references only

Serialization

```
1 void PluginProcessor::getStateInformation(juce::MemoryBlock& destData) {
2     juce::MemoryOutputStream outputStream{destData, true};
3     // write parameters to outputStream
4 }
5
6 void PluginProcessor::setStateInformation(const void* data, int sizeInBytes) {
7     juce::MemoryInputStream inputStream{data, static_cast<size_t>(sizeInBytes), false};
8     // read parameters from input stream
9 }
```

Serialization

parameter IDs and human-readable values → data format (e.g., JSON)

```
1  juce::AudioParameterFloat& floatParam;  
2  juce::AudioParameterInt& intParam;  
3  juce::AudioParameterBool& boolParam;  
4  juce::AudioParameterChoice& choiceParam;  
5  // ⬇  
6  struct IdAndValue {  
7      std::string id;  
8      std::variant<float,int,bool,std::string> value;  
9  };  
10 std::vector<IdAndValue> serializedParameters;  
11 // ⬇  
12 // serialization to file and to stream
```

Serialization

parameter IDs and human-readable values → data format (e.g., JSON)

Example for gain

```
1  juce::AudioParameterFloat& gain; // value: 1.f
2  // ⬇
3  struct IdAndValue {
4      std::string id; // "gain"
5      std::variant<float,int,bool,std::string> value; // 1.f
6  };
7  // ⬇
```

```
1  {
2      "parameters": [
3          {
4              "id": "gain",
5              "value": 1.0
6          }
7      ]
8  }
```

Parameters via references only

Serialization

```
1  std::vector<IdAndValue> serializeParameters(const juce::AudioParameterFloat& gain) {  
2      return {  
3          { .id = gain.getParameterID().toStdString(), .value = gain.get() }  
4      };  
5  }
```

Parameters via references only

Serialization

```
1  std::vector<IdAndValue> serializeParameters(  
2      const juce::AudioParameterFloat& floatParam,  
3      const juce::AudioParameterInt& intParam,  
4      const juce::AudioParameterBool& boolParam,  
5      const juce::AudioParameterChoice& choiceParam) {  
6  
7      return {  
8          { .id = floatParam.getParameterID().toStdString(), .value = floatParam.get() }  
9          { .id = intParam.getParameterID().toStdString(), .value = intParam.get() }  
10         { .id = boolParam.getParameterID().toStdString(), .value = boolParam.get() }  
11         { .id = choiceParam.getParameterID().toStdString(),  
12           .value = choiceParam.getCurrentChoiceName().toStdString() } // 🙌  
13     };  
14 }
```

Parameters via references only

Serialization

```
1  {  
2    "parameters": [  
3      {  
4        "id": "floatParam",  
5        "value": 1.0  
6      },  
7      {  
8        "id": "choiceParam",  
9        "value": "choice 1"  
10     },  
11     // ...  
12   ]  
13 }
```

Parameters via references only

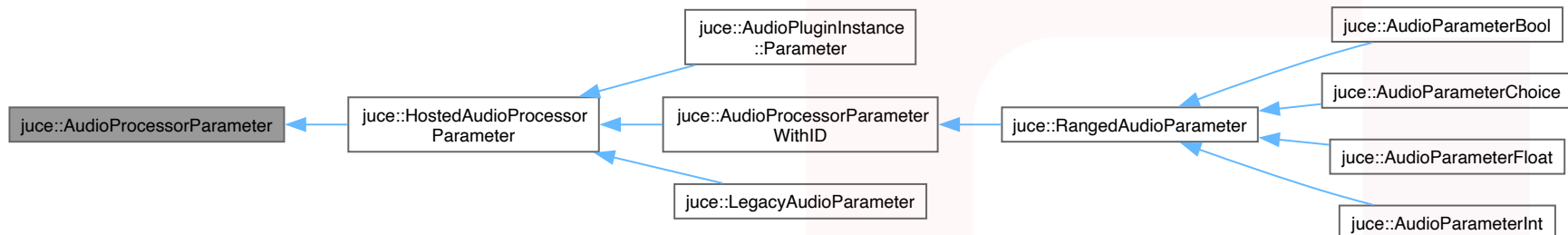
Pros

- Full type safety
- Easy access to singular parameters
 - `processBlock()`
 - `PluginEditor`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Cons

- "Manual" serialization code
 - Adding new parameters requires updating `serializeParameters()` $\implies$ error-prone

Parameters via a vector of base class pointers



```
1  class PluginProcessor : public juce::AudioProcessor {
2      // ...
3      juce::AudioParameterFloat& floatParam; // for audio processing & UI
4      // remaining parameters ...
5      std::vector<juce::RangedAudioParameter*> parameters;
6  };
7
```

Parameters via a vector of base class pointers

```
1  class JUCE_API RangedAudioParameter : public AudioProcessorParameterWithID {
2  public:
3      float convertTo0to1(float v) const noexcept;
4      float convertFrom0to1(float v) const noexcept;
5
6      // inherited:
7      /* Hosts will expect the value returned to be between 0 and 1.0. */
8      float getValue() const;
9      /* The value passed will be between 0 and 1.0. */
10     void setValue(float newValue);
11 };
12
13 class JUCE_API AudioParameterBool : public RangedAudioParameter {
14 public:
15     bool get() const noexcept;
16     operator bool() const noexcept;
17     // ...
18 };
19 class JUCE_API AudioParameterChoice : public RangedAudioParameter {
20 public:
21     String getCurrentChoiceName() const;
22     // ...
23 };
```

Parameters via a vector of base class pointers

```
1  std::vector<IdAndValue> serializeParameters(const std::vector<juce::RangedAudioParameter*>& parameters) {
2      return
3          parameters
4          | std::views::transform(
5              [](auto const* p) → IdAndValue {
6                  return {
7                      .id = p→getParameterID().toStdString(),
8                      .value = p→convertFrom0to1(p→getValue())
9                  };
10             })
11          | std::ranges::to<std::vector>();
12 }
```

Parameters via a vector of base class pointers

```
1  {  
2    "parameters": [  
3      { "id": "envelope.adbdr.attack.curve", "value": 1.0 },  
4      { "id": "envelope.adbdr.attack.time", "value": 30.0 },  
5      { "id": "envelope.adbdr.breakLevel", "value": 0.6000000238418579 },  
6      { "id": "envelope.adbdr.decay1.curve", "value": 1.0 },  
7      { "id": "envelope.adbdr.decay1.time", "value": 20.0 },  
8      { "id": "envelope.adbdr.decay2.curve", "value": 1.0 },  
9      { "id": "envelope.adbdr.decay2.time", "value": 20000.0 },  
10     { "id": "envelope.adbdr.release.curve", "value": 1.0 },  
11     { "id": "envelope.adbdr.release.time", "value": 300.0 },  
12     { "id": "filter.contourAmount", "value": 1.0 },  
13     { "id": "filter.cutoff", "value": 1.0 },  
14     { "id": "filter.passbandAttenuation", "value": 0.0 },  
15     { "id": "filter.resonance", "value": 0.0 },  
16     { "id": "frequencyOfA4", "value": 440.0 },  
17     { "id": "output.volume", "value": 1.0 },  
18     { "id": "pitchBend.semitonesDown", "value": -12.0 },  
19     { "id": "pitchBend.semitonesUp", "value": 2.0 },  
20     { "id": "waveshaper.autoMakeUpGain", "value": 0.0 }  
21   ]  
22 }
```

Parameters via a vector of base class pointers

Pros

- "Automatic" serialization code
- Easy access to singular parameters
 - `processBlock()`
 - `PluginEditor`
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization

Cons

- No type information during serialization
 - → human-unfriendly format



Summary so far

1. We can treat plugin parameters individually using only concrete

`juce::AudioParameterFloat|Bool|Int|Choice` classes $\implies$ We cannot (easily) define operations on a collection of parameters

2. We can treat plugin parameters as a collection of `juce::RangedAudioParameter`s $\implies$ We lose type information

Summary so far

1. Individual parameters: interpretability
2. Parameter collection: extensibility



How can we treat parameters as a collection without losing type information? 🤔

- ~~change JUCE~~
- `dynamic_cast` on `std::vector<juce::RangedAudioParameter*>` elements?
 - verbose, error-prone, slow, not general
- `std::vector<std::variant<juce::AudioParameterFloat*, juce::AudioParameterInt*, juce::AudioParameterBool*, juce::AudioParameterChoice*>>`
 - limits the set of supported parameter types
- Extending each `juce::AudioParameter_` class with a base class controlled by us
 - manual, unusable as a library feature
- Type Erasure?

Type-Erased Parameters

Collection of parameters

```
1  class TypeErasedParameter;  
2  
3  std::vector<TypeErasedParameter> parameters;
```

TypeErasedParameter

```

1  class TypeErasedParameter {
2  public:
3      template <class Parameter>
4      TypeErasedParameter(Parameter& p) : _impl{std::make_unique<ParameterModel<Parameter>>(p)} {}
5
6  private:
7      class ParameterConcept {
8      public:
9          virtual ~ParameterConcept() = default;
10     };
11
12     template <class Parameter>
13     class ParameterModel : public ParameterConcept {
14     public:
15         ParameterModel(Parameter& p) : _p{p} {}
16     private:
17         Parameter& _p;
18     };
19
20     std::unique_ptr<ParameterConcept> _impl;
21 };
22
23 std::vector<TypeErasedParameter> parameters;

```

Operations

```

1  IdAndValue serializeImpl(juce::AudioParameterFloat& p);
2  IdAndValue serializeImpl(juce::AudioParameterBool& p);
3  // ...
4  class TypeErasedParameter {
5  public:
6      // ...
7      IdAndValue serialize() const { return _impl->serialize(); }
8  private:
9      class ParameterConcept {
10     public:
11         virtual ~ParameterConcept() = default;
12         virtual IdAndValue serialize() const = 0;
13     };
14     template <class Parameter>
15     class ParameterModel : public ParameterConcept {
16     public:
17         // ...
18         IdAndValue serialize() const override { return serializeImpl(_p); }
19
20     private:
21         Parameter& _p;
22     };
23     std::unique_ptr<ParameterConcept> _impl;
24 };

```

Serialization

```
1  template <typename T>
2  IdAndValue serializeImpl(const T& parameter) {
3      return { parameter.getParameterID().toString(), parameter.get() };
4  }
5
6  template <>
7  IdAndValue serializeImpl<juce::AudioParameterChoice>(const juce::AudioParameterChoice& parameter) {
8      return { parameter.getParameterID().toString(), parameter.getCurrentChoiceName().toString() };
9  }
```

Serialization

```
1  std::vector<IdAndValue> serializeParameters(const std::vector<TypeErasedParameter>& parameters) {  
2      return  
3          parameters  
4              | std::views::transform(  
5                  [](const auto& p) { return p.serialize(); })  
6              | std::ranges::to<std::vector>();  
7  }
```

`TypeErasedParameter` and `serializeParameters()` don't depend on any JUCE class anymore!

The only requirement for type `T` contained in `TypeErasedParameter` is the presence of the `serializeImpl(T const&)` overload.

Operations via a Visitor

```

1  struct Visitor {
2      virtual ~Visitor = default;
3      virtual void visit(juce::AudioParameterFloat& p) = 0;
4      virtual void visit(juce::AudioParameterBool& p) = 0;
5      // ...
6  };
7
8  class TypeErasedParameter {
9  public:
10     // ...
11     void accept(Visitor& v) { _impl->accept(v); }
12 private:
13     class ParameterConcept {
14     public:
15         virtual ~ParameterConcept() = default;
16         virtual void accept(Visitor&) = 0;
17     };
18     template <class Parameter>
19     class ParameterModel : public ParameterConcept {
20     public:
21         // ...
22         void accept(Visitor& v) override { v.visit(_p); }
23
24     private:
25         Parameter& _p;
    
```

What if we want to support custom parameter classes?

⇒ make `TypeErasedParameter` templated on the `Visitor` class.

What if we want to support custom parameter classes?

```
1  template <class VisitorBase>
2  class TypeErasedParameter {
3  public:
4      void accept(VisitorBase& v) {/* ... */}
5      // ...
6  };
```

What if we want to support custom parameter classes?

A good default

```
1  struct JuceParameterVisitor {  
2      virtual ~JuceParameterVisitor() = default;  
3      virtual void visit(juce::AudioParameterBool&) = 0;  
4      virtual void visit(juce::AudioParameterFloat&) = 0;  
5      virtual void visit(juce::AudioParameterInt&) = 0;  
6      virtual void visit(juce::AudioParameterChoice&) = 0;  
7  };
```

Collection of TypeErasedParameters

```
1 std::vector<TypeErasedParameter> parameters;
```

Builder

```

1  class ParameterBuilder {
2  public:
3      template <class P, class ... Args>
4      P& add(Args&& ... args) {
5          auto parameter = std::make_unique<P>(std::forward<Args>(args) ... );
6          auto& ref = *parameter;
7          _parameterReferences.emplace_back(ref);
8          _parameters.push_back(std::move(parameter));
9          return ref;
10     }
11
12     std::vector<TypeErasedParameter> build(juce::AudioProcessor& p) && {
13         for (auto&& parameter : _parameters) {
14             p.addParameter(parameter.release());
15         }
16         return std::move(_parameterReferences);
17     }
18
19 private:
20     std::vector<std::unique_ptr<juce::AudioProcessorParameter>> _parameters;
21     std::vector<TypeErasedParameter> _parameterReferences;
22 };

```

Usage

```

1  class PluginProcessor : public juce::AudioProcessor {
2  public:
3      explicit PluginProcessor(
4          ParameterBuilder builder = {})
5          : floatParam{builder.add<juce::AudioParameterFloat>(
6              "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f)},
7            boolParam{builder.add<juce::AudioParameterBool>(
8              "boolParam", "Bool Param", true)},
9            intParam{builder.add<juce::AudioParameterInt>(
10             "intParam", "Int Param", 5, 10, 6)},
11            choiceParam{builder.add<juce::AudioParameterChoice>(
12              "choiceParam", "Choice Param", juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)},
13            parameters{std::move(builder).build(*this)} {}
14
15      // ...
16  private:
17      juce::AudioParameterFloat& floatParam;
18      juce::AudioParameterBool& boolParam;
19      juce::AudioParameterInt& intParam;
20      juce::AudioParameterChoice& choiceParam;
21      std::vector<TypeErasedParameter> parameters;
22  };

```

Type-erased parameter leveraging STL

```
1  IdAndValue serializeImpl(juce::AudioParameterFloat& p);
2  IdAndValue serializeImpl(juce::AudioParameterBool& p);
3  // ...
4  class AnyParameter {
5  public:
6      template <typename T>
7      AnyParameter(const T& parameterRef)
8          : parameterPtr{&parameterRef},
9            serializer{[&]() → IdAndValue {
10                auto ptr = std::any_cast<const T*>(parameterPtr); // reification
11                return serializeImpl(*ptr);
12            }} {}
13
14      IdAndValue serialize() const { return serializer(); }
15
16  private:
17      std::any parameterPtr;
18      std::function<IdAndValue()> serializer;
19  };
```

Type-erased parameter leveraging STL

```
1  class AnyParameter {
2  public:
3      template <typename T>
4      AnyParameter(const T& parameterRef)
5          : serializer{[&](){ return IdAndValue {
6              return serializeImpl(parameterRef);
7          }} {}
8
9      IdAndValue serialize() const { return serializer(); }
10
11 private:
12     std::copyable_function<IdAndValue()> serializer; // C++ 26
13 };
```

tc::any_range_ref

```

1  template <typename T>
2  struct any_range_ref {
3      template <typename Rng>
4      any_range_ref(Rng&& rng) noexcept
5          : m_pfuncTypeErased(
6              [](tc::any_ref anyrefRng,
7                  tc::function_ref<tc::break_or_continue (T) noexcept> fn) noexcept → tc::break_or_continue {
8                  return tc::for_each(anyrefRng.get_ref<std::remove_reference_t<Rng>>(), fn);
9              }
10         )
11         , m_anyrefRng(tc::as_lvalue(rng))
12     {}
13
14     tc::break_or_continue
15     operator()(tc::function_ref<tc::break_or_continue (T) noexcept> fn) const& noexcept {
16         return m_pfuncTypeErased(m_anyrefRng, fn);
17     }
18
19 private:
20     tc::type_erased_function_ptr<
21         /*bNoExcept*/true, tc::break_or_continue, tc::function_ref<tc::break_or_continue (T) noexcept>>
22         m_pfuncTypeErased;
23     tc::any_ref m_anyrefRng;
24 };

```

Another example: `tc::any_range_ref`

```
1 auto stringify_concat(tc::any_range_ref<int> anyrngref) {  
2     return tc::make<tc::string>(tc::join_with_separator(" ", tc::transform(anyrngref, tc_fn(tc::as_dec))));  
3 }  
4  
5 assert("0, 1, 2, 3, 4, 5" == (stringify_concat(std::vector<int>{0, 1, 2, 3, 4, 5})));  
6 assert("0, 1, 2, 3, 4, 5" == (stringify_concat(std::list<int>{0, 1, 2, 3, 4, 5})));
```

`tc::any_range_ref`, `tc::function_ref`: <https://github.com/think-cell/think-cell-library/blob/main/tc/base/ref.h>

Type-erased parameters

Pros

- Full type safety
- Easy access to singular parameters
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization
- **Automatic serialization of all parameters**
- (if using Visitor) Easy to add new operations on all parameters

Cons

- We need to store an additional vector

Tuple with parameter references

```

1  template <typename Parameter, typename ... Args>
2  Parameter& createAndAddParameter(juce::AudioProcessor& processor, Args&& ... args) { /* ... */ }
3
4  class PluginProcessor : public juce::AudioProcessor {
5  public:
6      PluginProcessor()
7          : parameters{createAndAddParameter<juce::AudioParameterFloat>(
8                      *this, "floatParam", "Float Param", juce::NormalisableRange{1.f, 10.f}, 5.f),
9                      createAndAddParameter<juce::AudioParameterBool>(
10                     *this, "boolParam", "Bool Param", true),
11                     createAndAddParameter<juce::AudioParameterInt>(
12                     *this, "intParam", "Int Param", 5, 10, 6),
13                     createAndAddParameter<juce::AudioParameterChoice>(
14                     *this, "choiceParam", "Choice Param",
15                     juce::StringArray{"choice 0", "choice 1", "choice 2"}, 1)} {}
16
17     std::tuple<juce::AudioParameterFloat&, juce::AudioParameterBool&,
18               juce::AudioParameterInt&, juce::AudioParameterChoice&> parameters;
19 };

```

Tuple with parameter references

Type-safe access

```
1  enum Parameters { floatParam, boolParam, intParam, choiceParam };  
2  
3  std::get<floatParam>(processor.parameters) = 2.f;  
4  std::get<boolParam>(processor.parameters) = false;  
5  std::get<intParam>(processor.parameters) = 7;  
6  std::get<choiceParam>(processor.parameters).getCurrentChoiceName();
```

Tuple with parameter references

Serialization

```
1  IdAndValue serializeImpl(juce::AudioParameterFloat& p);
2  IdAndValue serializeImpl(juce::AudioParameterBool& p);
3  // ...
4  template <typename ... Ts>
5  std::vector<IdAndValue> serializeParameters(const std::tuple<Ts ...>& parameters) {
6      std::vector<IdAndValue> result;
7
8      std::apply(
9          [&result](const Ts& ... parameterRefs) {
10              (
11                  (result.push_back(serializeImpl(parameterRefs))),
12                  ... );
13              },
14          parameters);
15
16      return result;
17  }
```

Tuple with parameter references

Pros

- Full type safety
- Easy access to singular parameters
- We can use any serialization format we like
- Serialization code can be reused for presets
- Easy UI attachments
- Possibility to add UI state serialization
- Automatic serialization of all parameters
- Easy to add new operations on all parameters
- Space-efficient

Cons

- Reading/debugging difficulty



Conclusion

- A good use of Type Erasure is to manage a collection of strongly-typed objects (that may or may not share a base class), when the types of the objects are not fixed in advance (or we have no control over those types) and we want to perform common actions for all elements of the collection
- $\Leftrightarrow$ Use Type Erasure for polymorphic behavior without inheritance or templates
- Use Type Erasure to physically decouple types and operations on those types
- $\Rightarrow$ overcome 3rd-party framework/library limitations

References

1. `TypeErasedParameter` with example serialization, <https://github.com/JanWilczek/wolfsound-dsp-utils>
 - `src/include/wolfsound/juce/wolfsound_ParameterHolder.hpp`
2. think-cell library: <https://github.com/think-cell/think-cell-library>
 - `tc::function_ref`, `tc::any_range_ref` : `tc/base/ref.h`
3. JUCE C++ framework source code, <https://github.com/juce-framework/JUCE>
4. Kevlin Henney, *Valued Conversions*, C++ Report July-August 2000
5. Sean Parent, *Inheritance Is the Base Class of Evil*, GoingNative 2013
6. Klaus Iglberger, *C++ Software Design: Design Principles and Patterns for High-Quality Software*, O'Reilly 2022
7. Fedor Pikus, *C++ Type Erasure Demystified*, C++Now 2024
8. Jan Wilczek & the JUCE team, *Official JUCE Audio Plugin Development Online Course*, <https://wolfsoundacademy.com/juce> (available for free)
9. Thanks to Daniel Lunow, Valentin Ziegler, and Roger Porta for helping me prepare this talk

Summary

1. Use Type Erasure for polymorphic behavior without inheritance or templates
2. Use Type Erasure to physically decouple types and operations on those types
 - $\implies$ overcome 3rd-party framework/library limitations
3. Check out our library at github.com/think-cell/think-cell-library
4. Apply for a C++ Developer position: hr@think-cell.com
5. Contact me via jwilczek_ext@think-cell.com

slides

